```
<div id="photos">loading photos...</div>
<script>
function loadPhotos() {
  var api_key = '<YOUR API KEY HERE>';
  var method = 'GET';
  var url = 'https://api.flickr.com/services/rest/?' +
      'method=flickr.people.getPublicPhotos&' +
      'user_id=32951986%40N05&' +
      'extras=url_q&format=json&nojsoncallback=1&' +
      'api_key=' + api_key;

  var xhr = new XMLHttpRequest();
  if (!('withCredentials' in xhr)) {
    alert('Browser does not support CORS.');
    return;
  }
  xhr.open(method, url);

  xhr.onerror = function() {
    alert('There was an error.');
  };

  xhr.onload = function() {
    var data = JSON.parse(xhr.responseText);
    if (data.stat == 'ok') {
      var photosDiv = document.getElementById('photos');
      photosDiv.innerHTML = '';
      var photos = data.photos.photo;
      for (var i = 0; i < photos.length; i++) {
        var img = document.createElement('img');
        img.src = photos[i].url_q;
        photosDiv.appendChild(img);
      }
    } else {
      alert(data.message);
    }
  };

  xhr.send();
}
</script>
</body>
</html>
```

Sets up request parameters

Sends request

Processes response

There are three parts to making an HTTP request:

- Sets up request parameters.
- Sends request.
- Processes response. (The code to process the response is defined before the request is actually sent, which is why `xhr.onload` comes before `xhr.send`.)

The rest of this section will examine how to use the `XMLHttpRequest` object during these three phases of the HTTP lifecycle. While the `XMLHttpRequest` object can be used to make either same-origin or cross-origin requests, this section will pay special attention to any CORS-specific behavior. Even if you have worked with the `XMLHttpRequest` object